1. PHẠM VI ĐỒ ÁN

Xây dựng một hệ thống streaming phân loại bình luận thành ba nhãn:

- **positive**
- **neutral**
- **negative**

> Sau đó so sánh chất lượng dự đoán và hiệu năng xử lý của pipeline.

---

2. NỘI DUNG CẦN THỰC HIỆN

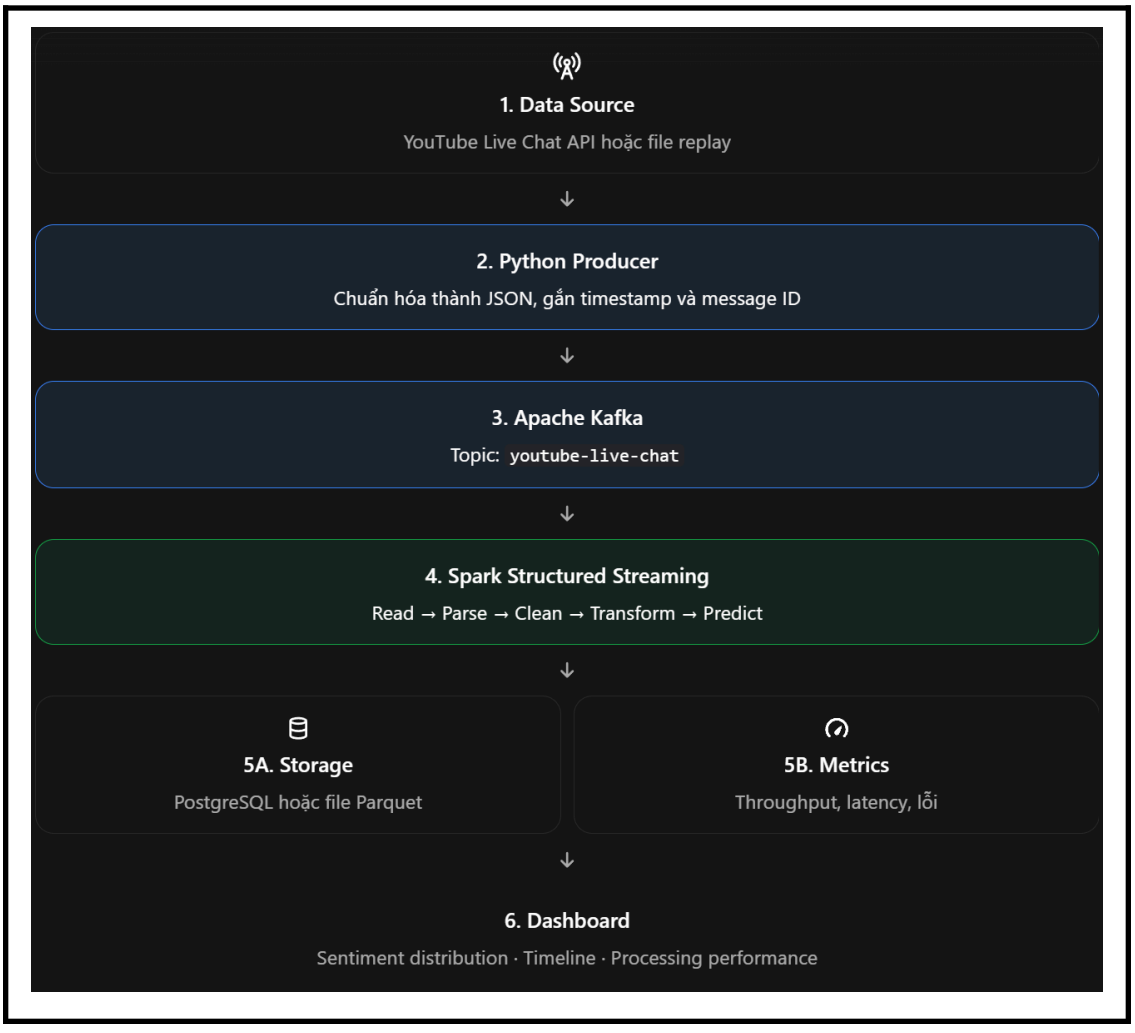
Data Ingestion	Thu thập chat thật từ YouTube Live API hoặc dùng dữ liệu replay để kiểm thử
Kafka	Producer gửi dữ liệu có schema thống nhất vào topic
Spark Streaming	Đọc Kafka, parse JSON, làm sạch và xử lý theo micro-batch
Sentiment model	Huấn luyện và tích hợp Logistic Regression; có baseline thứ hai để so sánh
Evaluation	Đánh giá Accuracy, Precision, Recall, F1-score và latency/throughput
Dashboard	Hiển thị cảm xúc theo thời gian và các chỉ số vận hành cơ bản

>> Những gì nên làm nếu còn thời gian

- Thêm Random Forest hoặc LSTM để mở rộng so sánh.
- Dùng Prometheus và Grafana để theo dõi throughput, processing time và tài nguyên.
- So sánh replay tốc độ bình thường với replay tốc độ cao.
- Kiểm thử khi Kafka tạm dừng, Spark khởi động lại hoặc xuất hiện tin nhắn lỗi.
- So sánh ảnh hưởng của tiền xử lý văn bản đến chất lượng mô hình.

---

3. PIPELINE + WORKFLOW DỰ KIẾN



4. TIÊU CHÍ HOÀN THÀNH

1. Dữ liệu có thể đi từ producer vào Kafka.
2. Spark đọc được dữ liệu và tạo ra dự đoán sentiment.
3. Kết quả có thể lưu, truy vấn và hiển thị.
4. Có thực nghiệm định lượng chất lượng mô hình lẫn hiệu năng streaming.
5. Một người khác có thể chạy lại hệ thống theo README và tái tạo được các kết quả chính.

1. Khả năng lấy dữ liệu

→ Sử dụng thư viện hỗ trợ **pytchat**. Xây dựng luồng cơ bản hệ thống như sau:



1. Tốc độ message ảnh hưởng gì đến Kafka và Spark?

Lúc chat tăng đột biến — burst

Producer gửi message nhanh hơn tốc độ Spark xử lý. Message tích tụ trong Kafka, độ trễ tăng. Nếu tình trạng này kéo dài, consumer có thể tụt lại phía sau (consumer lag).

Lúc chat chậm

Spark có thể nhận các micro-batch rất ít dữ liệu hoặc batch rỗng. Không nhất thiết có lỗi; nhưng chi phí xử lý mỗi batch vẫn có thể tồn tại.

Message đến trễ

Nếu chỉ xử lý theo thời gian nhận, nhóm có thể không phản ánh đúng thời điểm chat thực sự xuất hiện. Nếu phân tích sentiment theo cửa sổ thời gian, cần xác định dùng timestamp của nguồn hay timestamp lúc hệ thống nhận được.

Thông số	Data Quality	ML/NLP	Kafka/Spark	Benchmark
Total messages	✓		✓	✓
Duplicate messages	✓	✓	✓	✓
Empty messages	✓	✓	✓	
Emoji rate		✓		
Language distribution		✓		
Message length	✓	✓	✓	✓
Timestamp distribution	✓		✓	✓
Schema completeness	✓		✓	✓

2. Cơ chế Replay dữ liệu từ File (Backup Generator)

3. Đề xuất JSON Schema cho Message

NOTE:

1. 2 chế độ replay Producer

- Fixed rate replay: 1, 10, 50, 100 msg/s (tìm tài liệu chọn số liệu phù hợp). Đánh giá khả năng chịu tải - dễ so sánh các lần benmark
- Timestamp-based replay: mô phỏng lại livestream. Đánh giá khả năng thích ứng với tải biến động - mô phỏng burst + khoảng lặng

<u>Những số liệu bám sát bài báo:</u>

- Kafka latency: Theo dõi thời gian message nằm trong hàng đợi hoặc các chỉ số queue/consumer lag phù hợp.
- Spark processing latency: Đo thời gian Spark hoàn thành một micro-batch
- End-to-end latency: Đo thời gian từ lúc Producer gửi message đến lúc kết quả được ghi thành công vào PostgreSQL

2. Train model từ dataset chuẩn

- Gặp vấn đề khác topic nên đánh giá có thể sai (positive, negative, neutral) => Có 2 phương án xử lý:

